

SULIT



SECP2523 DATABASE

SESSION 2024/2025 - SEMESTER 1

ALTERNATIVE ASSESSMENT REPORT: PHASE 2

Name	LIM YU HAN
IC No. / Matric No.	A23CS0241
Year / Program	2/SECPH
Section	02
Lecturer Name	DR. SEAH CHOON SEAN
Case Study	Saving Account Management Module
System Name	Online KADA System
Group Name	TechMedico

Table of Contents

SECTION C: SQL AND INTERFACE IMPLEMENTATION 3

3.1 SQL Statement (DDL & DML)..... 3

3.2 Interface Implementation 10

SECTION D: CONCLUSION AND REFLECTION 15

4.1 Conclusion 15

4.2 Reflection 15

SECTION C: SQL AND INTERFACE IMPLEMENTATION

3.1 SQL Statement (DDL & DML)

a) SQL (DDL) Statement

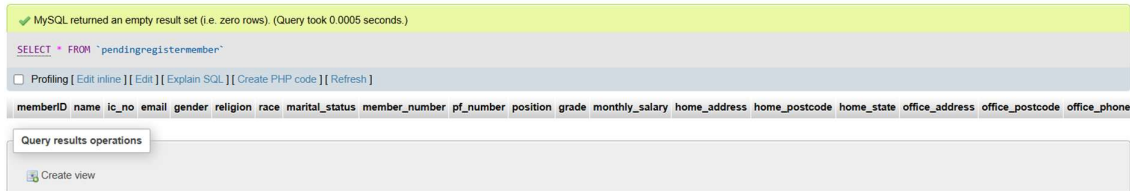
1. Create pendingregistermember table.

```
CREATE TABLE pendingregistermember (  
    memberID INT NOT NULL,  
    name VARCHAR(255) NOT NULL,  
    ic_no VARCHAR(22) PRIMARY KEY,  
    email VARCHAR(30) NOT NULL,  
    gender ENUM('male', 'female') NOT NULL,  
    religion VARCHAR(50) NOT NULL,  
    race VARCHAR(50) NOT NULL,  
    marital_status ENUM('single', 'married', 'divorced', 'widowed') NOT NULL,  
    member_number VARCHAR(50) NOT NULL,  
    pf_number VARCHAR(50) NOT NULL,  
    position VARCHAR(100) NOT NULL,  
    grade VARCHAR(50) NOT NULL,  
    monthly_salary DECIMAL(10, 2) NOT NULL,  
    home_address TEXT NOT NULL,  
    home_postcode VARCHAR(10) NOT NULL,  
    home_state VARCHAR(50) NOT NULL,  
    office_address TEXT NOT NULL,  
    office_postcode VARCHAR(10) NOT NULL,  
    office_phone VARCHAR(20) NOT NULL,  
    home_phone VARCHAR(20) NOT NULL,  
    fax VARCHAR(20),  
    registration_fee DECIMAL(10, 2) NOT NULL,  
    share_capital DECIMAL(10, 2) NOT NULL,  
    fee_capital DECIMAL(10, 2) NOT NULL,  
    deposit_funds DECIMAL(10, 2) NOT NULL,  
    welfare_fund DECIMAL(10, 2) NOT NULL,  
    fixed_deposit DECIMAL(10, 2) NOT NULL,  
    other_contributions TEXT,
```

```

        created_at TIMESTAMP,
        updated_at TIMESTAMP,
        status ENUM('pending', 'approve', 'rejected') NOT NULL,
        admin_remark TEXT
    );

```

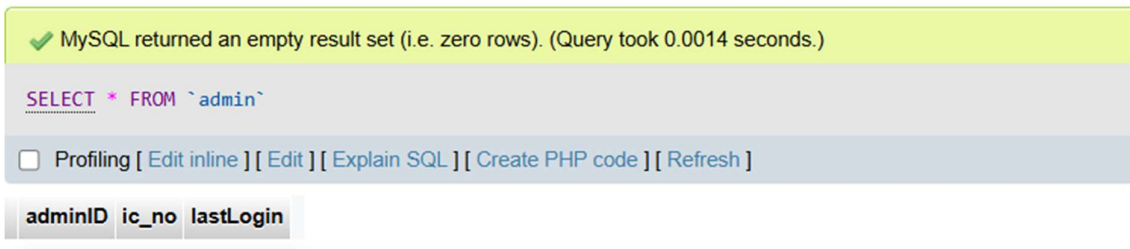


2. Create admin table.

```

CREATE TABLE admin (
    adminID INT NOT NULL,
    ic_no VARCHAR(22) PRIMARY KEY,
    lastLogin DATETIME
);

```



3. Create saving_accounts table.

```

CREATE TABLE saving_accounts (
    accountID INT AUTO_INCREMENT PRIMARY KEY,
    ic_no VARCHAR(22) NOT NULL,
    account_number VARCHAR(20) UNIQUE NOT NULL,
    balance DECIMAL(10, 2) NOT NULL,
    status ENUM('active', 'inactive') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
);

```

```

has_paid_one_time_fees BOOLEAN DEFAULT FALSE,
FOREIGN KEY (ic_no) REFERENCES pendingregistermember (ic_no)
);

```



4. Create saving_transactions table.

```

CREATE TABLE saving_transactions (
    transactionID INT AUTO_INCREMENT PRIMARY KEY,
    accountID INT NOT NULL,
    transaction_type ENUM('deposit', 'transfer') NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    payment_method ENUM('fpx', 'card'),
    deposit_bank VARCHAR(50),
    card_type VARCHAR(20),
    bank_name VARCHAR(50),
    bank_account VARCHAR(20),
    transfer_purpose VARCHAR(50),
    description TEXT NOT NULL,
    admin_remark TEXT,
    processed_by INT,
    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending',
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);

```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0020 seconds.)														
SELECT * FROM `saving_transactions`														
☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]														
transactionID	accountID	transaction_type	amount	payment_method	deposit_bank	card_type	bank_name	bank_account	transfer_purpose	description	admin_remark	processed_by	processed_at	status
transaction_date														
Query results operations														
Create view														

5. Add a foreign key constraint to the saving_transactions table.

```
ALTER TABLE saving_transactions
ADD CONSTRAINT FK_accountID
FOREIGN KEY (accountID) REFERENCES saving_accounts (accountID);
```

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0386 seconds.)														
ALTER TABLE saving_transactions ADD CONSTRAINT FK_accountID FOREIGN KEY (accountID) REFERENCES saving_accounts (accountID);														
[Edit inline] [Edit] [Create PHP code]														

b) SQL (DML) Statement

1. Insert a new record into pendingregistermember table.

```
INSERT INTO pendingregistermember (memberID, name, ic_no, email, gender, religion,
race, marital_status, member_number, pf_number, position, grade, monthly_salary,
home_address, home_postcode, office_address, office_postcode,
office_phone, home_phone, registration_fee, share_capital, fee_capital,
deposit_funds, welfare_fund, fixed_deposit, created_at, updated_at, status)
VALUES
(1, 'YY', '990101-01-1234', 'admad11@gmail.com', 'male', 'Christianity', 'Chinese',
'single', 'MEM001', 'PF001', 'Manager', 'A1', 5000.00, '123 Street Name', '12345',
'State A', '456 Office Name', '54321', '03-12345678', '012-3456789', 50.00, 500.00,
100.00, 200.00, 50.00, 1000.00, NOW(), NOW(), 'pending');
```

memberID	name	ic_no	email	gender	religion	race	marital_status	member_number	pf_number	position	grade	monthly_salary	home_address	home_postcode
1	YY	990101-01-1234	admad11@gmail.com	male	Christianity	Chinese	single	MEM001	PF001	Manager	A1	5000.00	123 Street Name	12345

2. Update the status of a member's application.

```
UPDATE pendingregistermember
SET status = 'approve', admin_remark = 'Application approved successfully'
WHERE ic_no = '990101-01-1234';
```

lice_postcode	office_phone	home_phone	fax	registration_fee	share_capital	fee_capital	deposit_funds	welfare_fund	fixed_deposit	other_contributions	created_at	updated_at	status	admin_remark
321	03-12345678	012-3456789	NULL	50.00	500.00	100.00	200.00	50.00	1000.00	NULL	2025-01-23 15:35:31	2025-01-23 15:35:31	approve	Application approved successfully

- Update the balance of a member's saving account after a deposit.

```
UPDATE saving_accounts
SET balance = balance + 500.00, updated_at = NOW()
WHERE accountID = 1;
```

accountID	ic_no	account_number	balance	status	created_at	updated_at	has_paid_one_time_fees
1	990101-01-1234	ACC001	1500.00	active	2025-01-23 15:56:35	2025-01-23 16:20:15	0

- Insert and then delete a pending transaction from saving_transactions by transactionID.

```
INSERT INTO saving_transactions (transactionID, accountID, transaction_type, amount,
bank_name, description, status)
VALUES (2, 1, 'transfer', 200, 'RHB Bank', 'transfer to customer', 'pending');
```

transactionID	accountID	transaction_type	amount	payment_method	deposit_bank	card_type	bank_name	bank_account	transfer_purpose	description	admin_remark
2	1	transfer	200.00	NULL	NULL	NULL	RHB Bank	NULL	NULL	transfer to customer	NULL

```
DELETE FROM saving_transactions
WHERE transactionID = 2 AND status = 'pending';
```

transactionID	accountID	transaction_type	amount	payment_method	deposit_bank	card_type	bank_name	bank_account	transfer_purpose	description	admin_remark	processed_by	processed_at	status	transaction_date
---------------	-----------	------------------	--------	----------------	--------------	-----------	-----------	--------------	------------------	-------------	--------------	--------------	--------------	--------	------------------

- Retrieve all saving accounts with their respective members' information.

```
SELECT sa.accountID, sa.account_number, sa.balance, pm.name, pm.ic_no, pm.email
FROM saving_accounts sa
INNER JOIN pendingregistermember pm
ON sa.ic_no = pm.ic_no;
```

accountID	account_number	balance	name	ic_no	email
2	ACC002	500.00	Jane	010101-01-1234	jane@gmail.com
1	ACC001	1500.00	YY	990101-01-1234	admad11@gmail.com

6. Retrieve all members and their saving accounts, including members who do not have saving accounts.

```
SELECT pm.name, pm.ic_no, sa.account_number, sa.balance
FROM pendingregistermember pm
LEFT JOIN saving_accounts sa
ON pm.ic_no = sa.ic_no;
```

name	ic_no	account_number	balance
Sun	001104-01-1234	NULL	NULL
Jane	010101-01-1234	ACC002	500.00
Rex	050221-01-1234	NULL	NULL
YY	990101-01-1234	ACC001	1500.00

7. Count total deposit transaction.

```
SELECT transaction_type, COUNT(transactionID) AS total_transactions
FROM saving_transactions
WHERE transaction_type = 'deposit'
GROUP BY transaction_type;
```

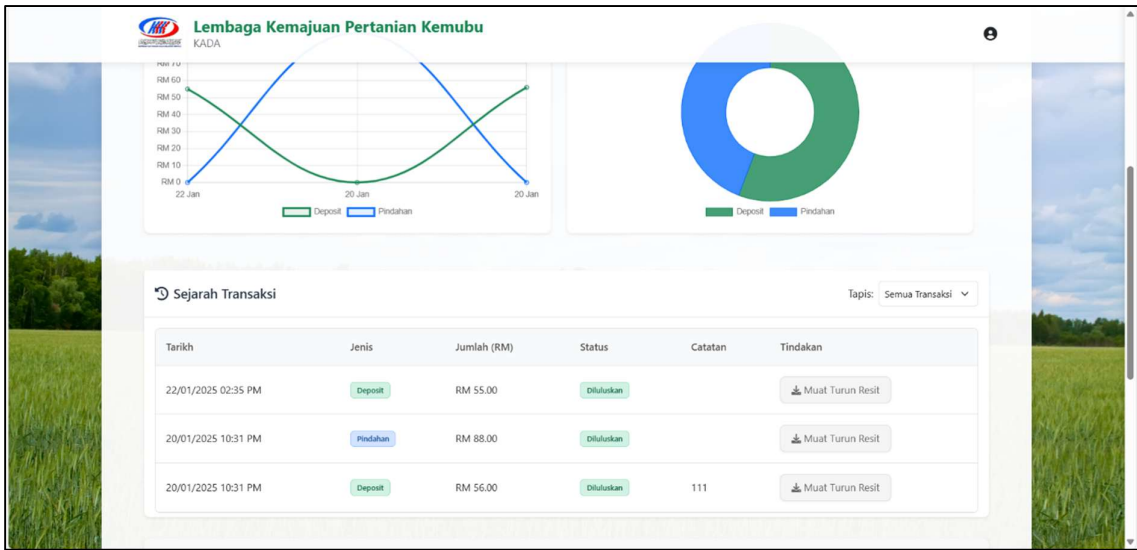
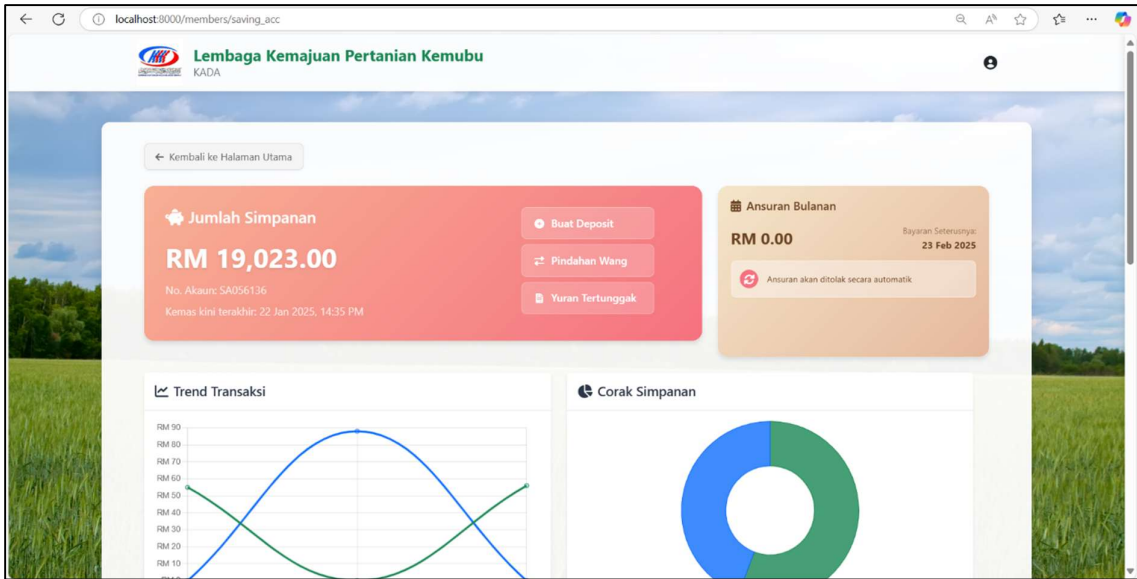
transaction_type	total_transactions
deposit	2

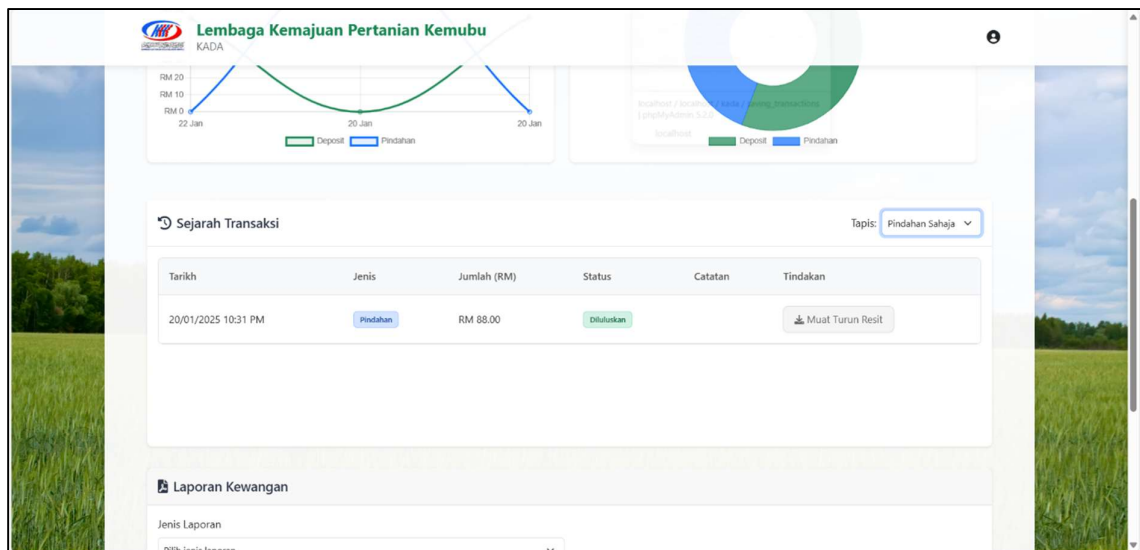
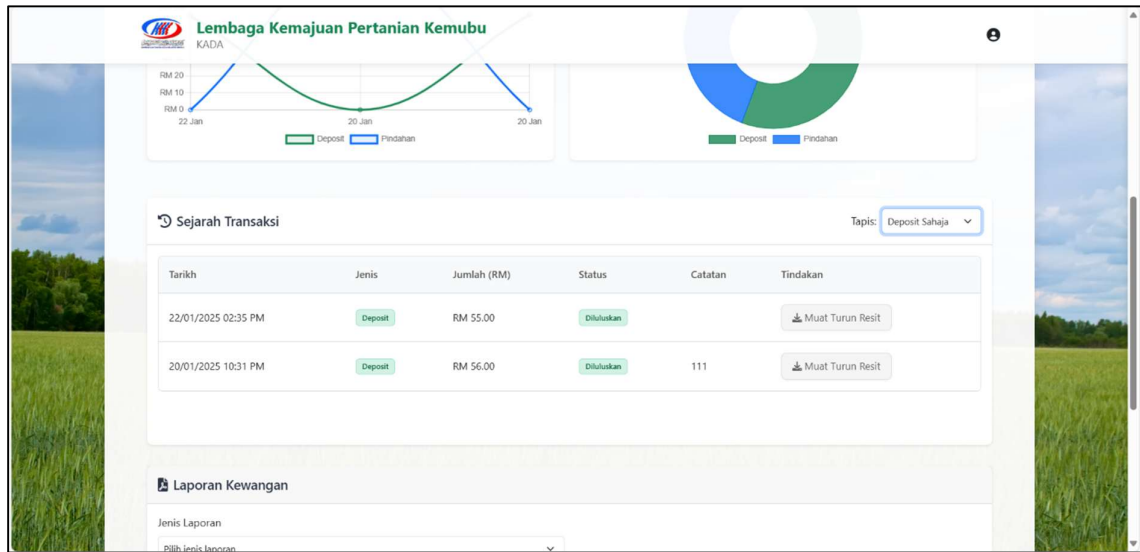
8. Select all transactions where the amount is greater than 30.

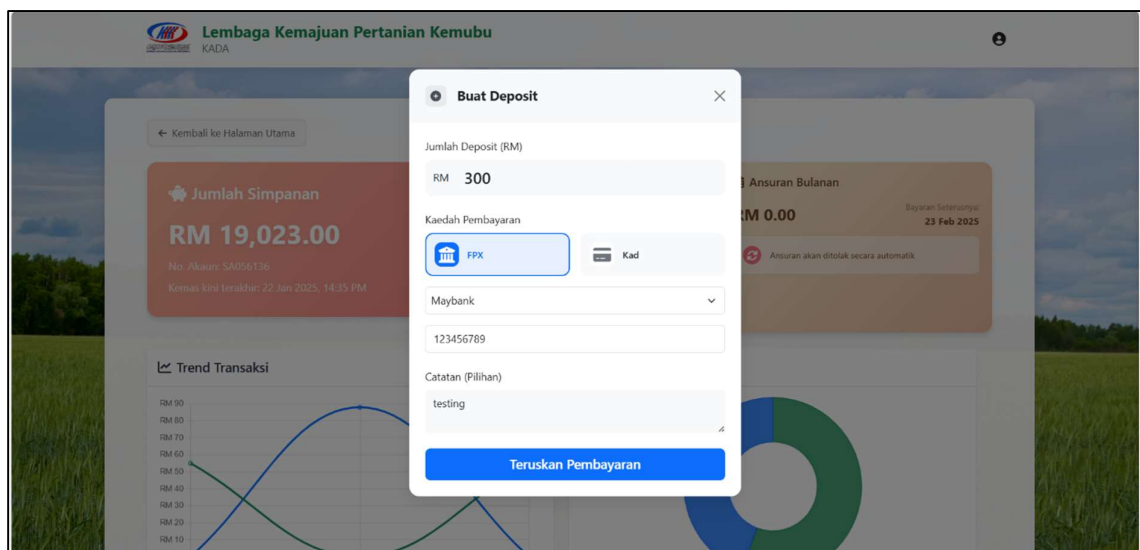
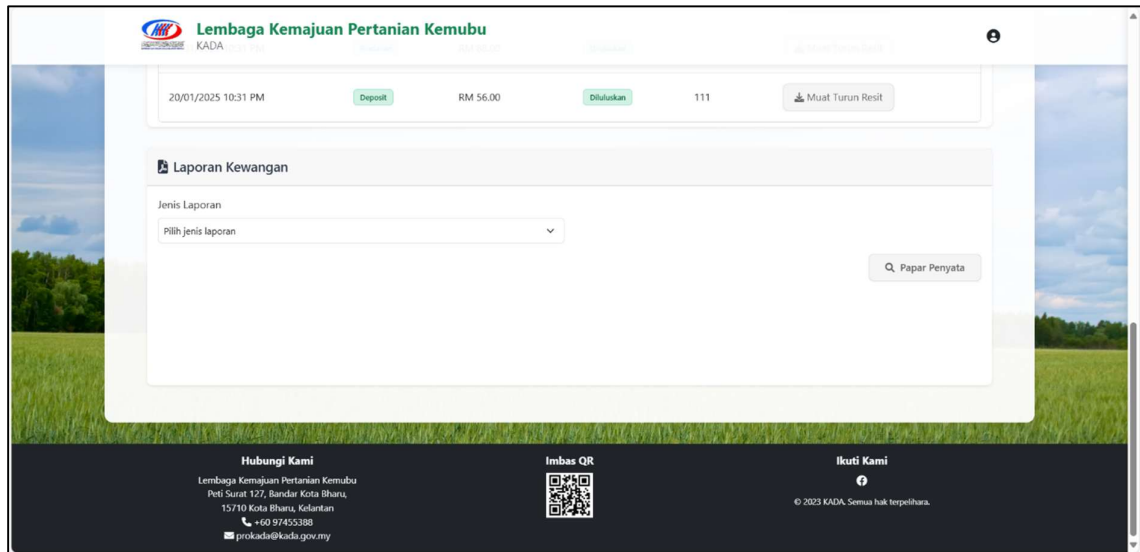
```
SELECT * FROM saving_transactions
WHERE amount > 30;
```

transactionID	accountID	transaction_type	amount	payment_method	deposit_bank	card_type	bank_name	bank_account	transfer_purpose	description	admin_remark	prc
1	1	deposit	300.00	NULL	Maybank	NULL	NULL	NULL	NULL	deposit	NULL	
2	1	deposit	50.00	NULL	Alliance Bank	NULL	NULL	NULL	NULL	deposit	NULL	
3	2	transfer	100.00	NULL	Public Bank	NULL	NULL	NULL	NULL	transfer	NULL	

3.2 Interface Implementation







**Lembaga Kemajuan Pertanian**
KADA

[← Kembali ke Halaman Utama](#)

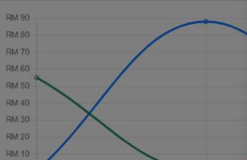
Jumlah Simpanan

RM 19,023.00

No. Akaun: SA056136

Kemas kini terakhir: 22 Jan 2025, 14:35 PM

Trend Transaksi



Buat Deposit

×

Jumlah Deposit (RM)

RM 50

Kaedah Pembayaran

FPX

Kad

Mastercard

15265489

05/25123

YY

Catatan (Pilihan)

testing

Teruskan Pembayaran

Ansuran Bulanan

Bayaran Seterusnya: 23 Feb 2025

RM 0.00

Ansuran akan ditolak secara automatik

**Lembaga Kemajuan Pertanian**
KADA

[← Kembali ke Halaman Utama](#)

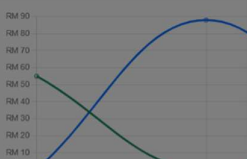
Jumlah Simpanan

RM 19,023.00

No. Akaun: SA056136

Kemas kini terakhir: 22 Jan 2025, 14:35 PM

Trend Transaksi



Pindahan Wang

×

Jumlah Pindahan (RM)

RM 100

Nama Bank

Public Bank

123456789

Tujuan Pindahan

Pembayaran

Pemindahan

Pendidikan

Lain-lain

Catatan (Pilihan)

testing

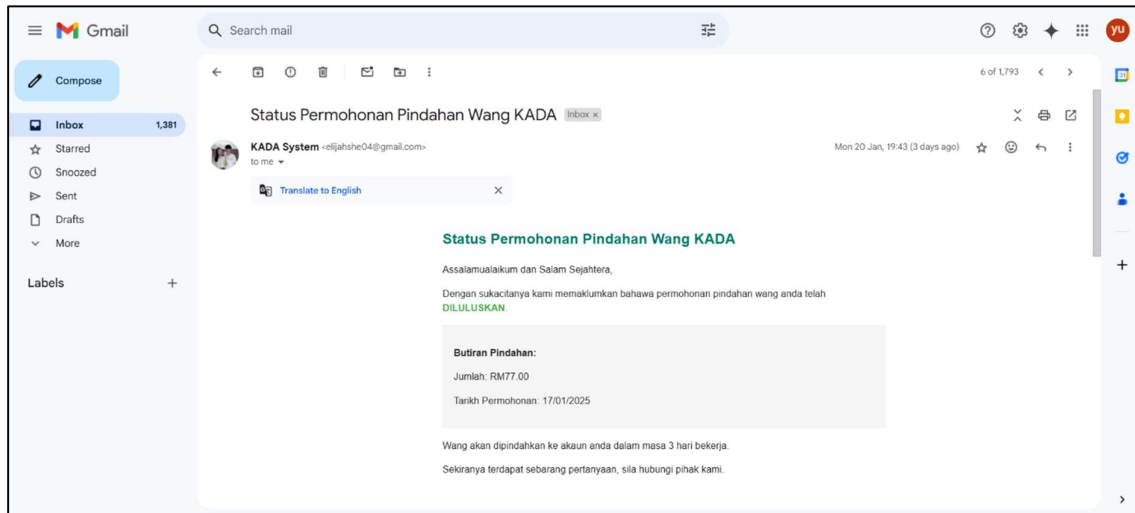
Teruskan Pembayaran

Ansuran Bulanan

Bayaran Seterusnya: 23 Feb 2025

RM 0.00

Ansuran akan ditolak secara automatik



SECTION D: CONCLUSION AND REFLECTION

4.1 Conclusion

One essential part of the cooperative's system that guarantees the effective administration of member savings accounts is the Saving Account Management Module. Creating and maintaining savings accounts, monitoring account balances and handling all kinds of transactions, including deposits and transfers, are all made possible by this module. The module guarantees data correctness and consistency while reducing manual labor by automating these procedures.

This module's capacity to improve user experience by incorporating features like transaction history monitoring and guaranteeing transparency for administrators and members alike is one of its main results. As the cooperative expands, the module can manage more accounts and transactions because it was created with scalability in mind.

Furthermore, the database schema was carefully designed to ensure compliance with normalization rules, up to 3NF, which improves data integrity, reduces redundancy, and optimizes query performance. The implementation also supports future integration with other modules, such as Financial Reports and Member Registration, creating a cohesive and interconnected system.

In conclusion, the Enhanced Saving Management Module not only fulfills the immediate operational needs of the cooperative but also lays the groundwork for future enhancements, such as real-time notifications, personalized transaction categorization, and dynamic financial reporting through graphs and charts. This module represents a significant step toward modernizing the cooperative's financial operations and enhancing the overall system's effectiveness.

4.2 Reflection

For the group project, I was tasked with the Saving Account Management Module and handled designing and implementing the database schema and operations for savings accounts. The most important tasks were establishing relationships, generating normalized database schemas, and writing SQL queries for basic operations like deposits, transfers, and transaction history. Database design was done based on the normalization rules up to the 3NF in order to maintain data integrity and prevent redundancy. Cooperative coordination was required to ensure that the database schema was compatible with the rest of the modules, such as Member Registration and Financial Reports. While the rest of the group members focused on their specific modules, we worked together to merge the database designs, thereby addressing the overall system requirements.

I felt nervous since the complexity of the database schema required proper planning and precise execution. There were frustrations in the process, like when synchronizing foreign key relationships across modules, especially ensuring compatibility with my module and other modules like Member Registration. We also changing and adding a lot of functions during the project as many things need to try only we can know it is suitable or not. Despite these challenges, I found the experience rewarding as it allowed me to apply theoretical database concepts to a real-world project. The collaborative environment further motivated me, as my groupmates were supportive and open to discussions about resolving design issues.

I also have both good and bad experience on this project. On the positive side, normalization up to 3NF was a huge benefit in eliminating redundancy of data and consistency of data across tables. It was challenging but rewarding to compose complex SQL queries for JOIN operations in order to derive meaningful information from more than one table. One challenge was to provide smooth flow

of data between modules, and this needed extra effort to maintain referential integrity. Besides, delays in schema matching due to reliance on other members' work caused some time management problems. On the whole, the experience demonstrated the significance of planning and communication in database design.

I chose this module for reflection because it had a significant impact on the entire system and allowed for the building of complex functionalities, such as data integrity management and logging of transactions. The experience gave me insight into the challenge of combining modules and database optimization in multi-developer environments. The primary focus area was making real-time updates to the financial information with performance and accuracy. This focus will guide future projects toward keeping functional as well as non-functional requirements in mind.

Through this project, I learned the importance of careful planning and team work habits towards achieving a successful result. I also learned to balance individual activities and group project, so I can excel in troubleshooting system problems efficiently. This project encouraged me to consider methods of improving performance in large systems and protecting data privacy, which I would like to follow through with in my subsequent projects.

In future, my primary focus in a project would be on enhancing collaboration with my groupmates during the early planning stages. By developing a shared ERD and addressing potential integration challenges at an earlier stage, we could have significantly reduced delays. Furthermore, I would delve into advanced SQL features to further streamline processes like transaction approvals and notifications. These changes would be facilitated by exposure to more learning materials, including design guidelines and database optimization tutorials on many database related websites. In the future, I will focus on improved planning and technical skills to ensure that database development is made easier and more effective.